Sistema de Gestión del Conocimiento de Softtek — Reporte de avance del proyecto

Oct 31, 2026

Repositorio del proyecto: https://github.com/regina120506/Avance_SYTAC

1. La empresa: Softtek

Softtek es una empresa mexicana privada de servicios de TI, fundada en diciembre de 1982 en Monterrey, con unos 15,000 empleados y 30 oficinas en Latinoamérica, Estados Unidos, Europa y Asia (Softtek Fast Facts). Introdujo el modelo de entrega Nearshore y opera centros globales de entrega en Monterrey, Ensenada, Guadalajara, Ciudad de México y Aguascalientes, además de Brasil, Argentina, Chile, Paraguay, Colombia, Perú, España, India y China. Con miles de consultores repartidos entre centros y proyectos de clientes, el conocimiento que necesita un equipo —la arquitectura de un cliente, un incidente anterior, una política— se pierde con facilidad; el sistema de gestión del conocimiento (KMS) propuesto lo centraliza, lo clasifica y lo hace buscable.

Caso de estudio académico. Los datos de la empresa provienen de la ficha pública de Softtek y de Wikipedia; los documentos de ejemplo, las instancias de la ontología y los resultados son ilustrativos, no datos internos de Softtek.

a. Tipo de organización. Empresa tecnológica: firma privada de servicios y consultoría de TI con sede en Monterrey, México, que atiende a clientes del Global 2000.

b. Actividades principales (servicios que lista la ficha oficial):

- Desarrollo ágil, modernización de aplicaciones, APIs y microservicios
- Aseguramiento de calidad (QA), DevOps e ingeniería de confiabilidad (SRE)
- Servicios de SAP y Salesforce
- Gestión de aplicaciones, gestión de nube y soporte a usuarios
- Consultoría de negocio digital, laboratorios de innovación, diseño de experiencia del cliente y automatización cognitiva

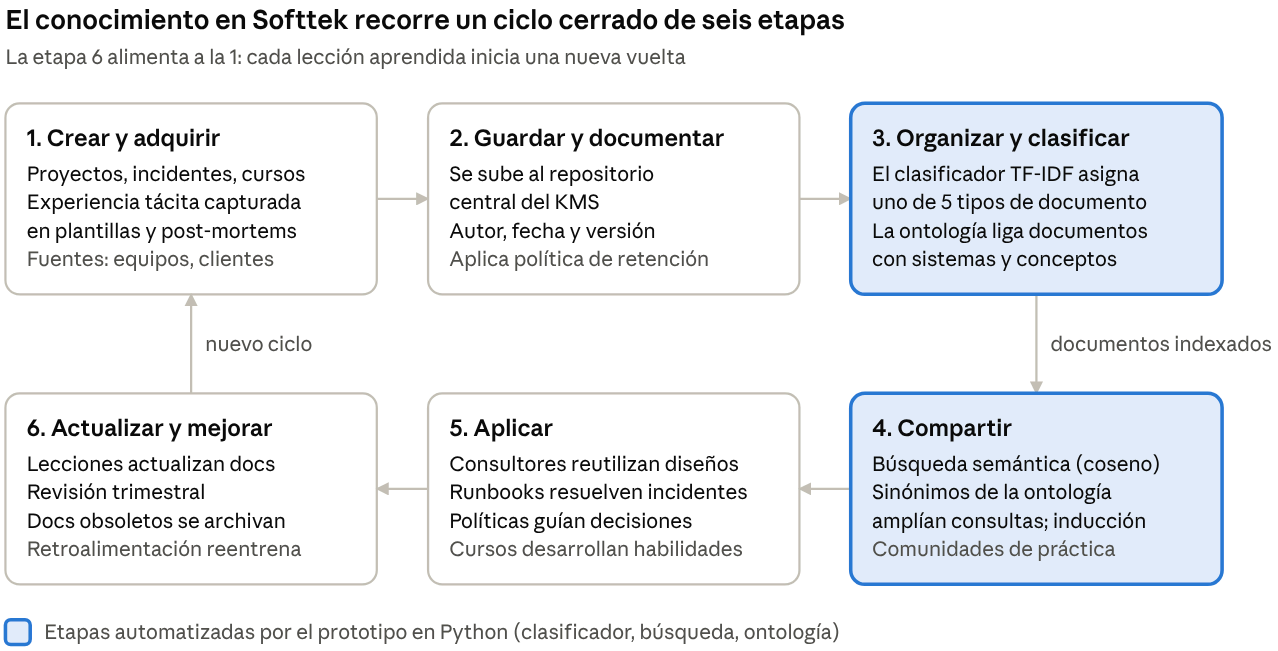
c. Tipos de conocimiento y documentos. Estos cinco tipos de documento son también las cinco categorías que predice el prototipo de clasificación.

Tipo de documento	Ejemplos	Responsable	Tipo de conocimiento
Documentación técnica	Referencias de API, diseños de arquitectura, guías de despliegue	Equipos de entrega	Explícito, técnico
Políticas y procedimientos	Política de seguridad, home office, reglas de viáticos	Cumplimiento / RH	Explícito, normativo
Reportes de proyecto	Reportes de avance, revisiones de sprint, lecciones aprendidas	Gerentes de proyecto	Explícito, experiencial
Manuales de capacitación	Guías de inducción, material de cursos, tutoriales	Área de talento / capacitación	Explícito, instruccional
Reportes de incidentes	Post-mortems de caídas, análisis de causa raíz, incidentes de seguridad	Equipos de operación / SRE	Experiencial, correctivo

Además de estos documentos, Softtek tiene conocimiento tácito —la experiencia de sus consultores, el contexto de cada cliente, el saber resolver fallas— que el ciclo de vida de la siguiente sección busca convertir en documentos explícitos y reutilizables.

2. Mapa del ciclo de vida del conocimiento

El conocimiento en Softtek recorre seis etapas en un ciclo cerrado; el prototipo automatiza las etapas 3 y 4, y su retroalimentación cierra la etapa 6.



Ciclo de vida del conocimiento en Softtek · 6 etapas

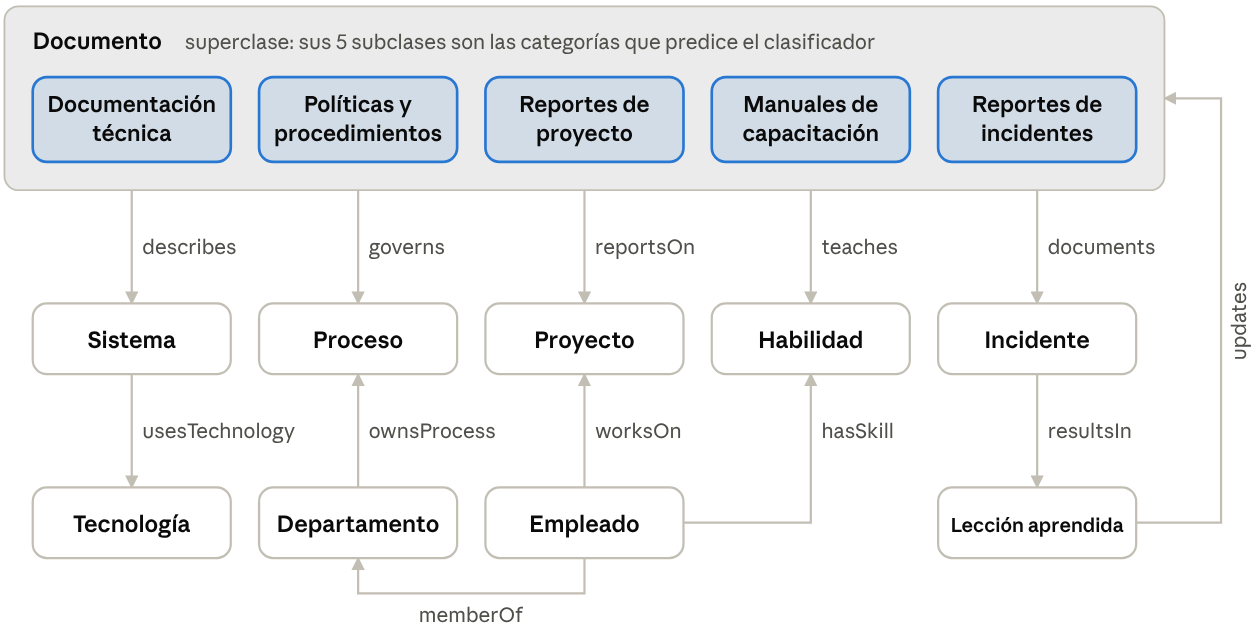
Ejemplo: la caída de la API de pagos (etapa 1) se documenta en el post-mortem IR01 (2), se clasifica como Reporte de incidente y se liga a la API de pagos (3), se encuentra al buscar "payment system went down" (4), orienta al ingeniero de guardia la siguiente vez (5), y su lección —automatizar la renovación de certificados— actualiza la guía de monitoreo TD07 (6).

3. Ontología del conocimiento organizacional

La ontología modela el conocimiento de Softtek con 13 clases del dominio, 5 clases de documento y 22 relaciones, escritas en OWL/SKOS Turtle (`ontology/softtek_ontology.ttl`, 372 tripletas), de modo que se puede consultar con SPARQL y el buscador la reutiliza.

Cada tipo de documento se ancla a un concepto central del dominio

Cajas rellenas: clases de documento; con contorno: conceptos del dominio; flechas: propiedades



También en el modelo: authoredBy, ownedBy, references, about (Concept), affects, hasRisk, deliveredFor

Ontología del KMS de Softtek · clases y propiedades principales

El ciclo de incidentes se ve en el grafo: un Reporte de incidente documenta un Incidente, que produce una Lección aprendida que actualiza otros documentos. Los nombres de las propiedades (describes, governs…) se dejan en inglés porque son los identificadores reales del archivo.

a. Conceptos principales del dominio: Organización, Departamento, Empleado, Cliente, Proyecto, Sistema/Servicio, Tecnología, Habilidad, Proceso de negocio, Incidente, Riesgo, Lección aprendida y Concepto (vocabulario de temas en SKOS).

b. Tipos de documento: `Document` con cinco subclases — TechnicalDocumentation, PolicyProcedure, ProjectReport, TrainingManual, IncidentReport.

c. Relaciones entre conceptos

Propiedad	De → A	Ejemplo de instancia
hasDepartment	Organización → Departamento	Softtek → Cloud Operations
memberOf	Empleado → Departamento	Ana (SRE) → Cloud Operations

Propiedad	De → A	Ejemplo de instancia
worksOn	Empleado → Proyecto	Luis → Proyecto plataforma retail
deliveredFor	Proyecto → Cliente	Plataforma retail → RetailCo
buildsSystem	Proyecto → Sistema	Plataforma retail → API de pagos
usesTechnology	Sistema → Tecnología	Servicio de inventario → PostgreSQL
hasSkill / skillIn	Empleado → Habilidad → Tecnología	Ana → Operación de Kubernetes → Kubernetes
affects	Incidente → Sistema	Caída de pagos → API de pagos
resultsIn	Incidente → Lección aprendida	Caída de pagos → Automatizar renovación TLS
hasRisk	Proyecto → Riesgo	Plataforma retail → Retraso del proveedor de pagos
ownsProcess	Departamento → Proceso	Cloud Operations → Gestión de incidentes
skos:broader	Concepto → Concepto	Outage → Incident

d. Relaciones entre documentos y conceptos

Propiedad	De → A	Ejemplo de instancia
describes	Documentación técnica → Sistema	TD01 Referencia de la API de pagos → API de pagos
governs	Políticas y procedimientos → Proceso	PP07 Gestión de cambios → Gestión de cambios
reportsOn	Reporte de proyecto → Proyecto	PR10 Variación de presupuesto → Banca móvil
teaches	Manual de capacitación → Habilidad	TM03 Curso de Kubernetes → Operación de Kubernetes
documents	Reporte de incidente → Incidente	IR01 Post-mortem → Caída de pagos

Propiedad	De → A	Ejemplo de instancia
about	Documento → Concepto	PP02 Política de home office → Trabajo remoto
authoredBy / ownedBy	Documento → Empleado / Departamento	PR01 → Luis / PMO
references	Documento → Documento	IR01 → TD01
updates	Lección aprendida → Documento	Agregar prueba de rendimiento → TD04, PP07

Cada concepto SKOS tiene etiquetas alternativas (por ejemplo, Outage: "went down", "unavailable"); el buscador las lee para ampliar las consultas (sección 5). Las etiquetas están en inglés porque los documentos de ejemplo lo están.

4. Prototipo de clasificación automática

El clasificador (`src/classifier.py`) asigna cada documento a uno de los cinco tipos; la regresión logística alcanza 0.90 de exactitud promedio en validación cruzada de 5 particiones, por encima de Naive Bayes con 0.82.

Cómo funciona

1. **Cargar documentos.** `load_documents()` lee `data/documents.csv` : 60 documentos de ejemplo etiquetados, 12 por categoría. El título y el cuerpo se unen en un solo campo `content` .
2. **Preprocesar.** `tokenize()` pasa a minúsculas, quita puntuación y dígitos, elimina stop words en inglés y aplica el stemmer de Porter (payments → payment, outage → outag).
3. **Vectorizar con TF-IDF.** `TfidfVectorizer` genera unigramas y bigramas con frecuencia de término sublineal; cada documento se vuelve un vector disperso de unos 2,180 términos ponderados.
4. **Entrenar.** Se entrenan dos pipelines de scikit-learn (TF-IDF + modelo) con una partición estratificada 75/25 (45 de entrenamiento / 15 de prueba): `MultinomialNB(alpha=0.1)` y `LogisticRegression(C=10)` .
5. **Evaluar.** Exactitud, precisión/recall/F1 por clase, matriz de confusión (PNG) y validación cruzada estratificada de 5 particiones.

6. **Clasificar.** El modelo final de regresión logística se reentrena con los 60 documentos, se guarda en `outputs/classifier.joblib` y se usa para etiquetar documentos nuevos (`data/new_documents.csv`) o cualquier texto pasado con `--text` . Devuelve la categoría y una confianza (la probabilidad más alta entre las clases).

El peso TF-IDF del término t en el documento d es:

```
\text{tfidf}(t,d) = (1 + \ln \text{tf}_{t,d}) \cdot \left( \ln\frac{1+N}
{1+\text{df}_t} + 1 \right)
```

donde N es el número de documentos y df el número de documentos que contienen t; después cada vector se normaliza con L2.

Ejecución

```
python src/classifier.py
python src/classifier.py --text "The VPN was unavailable for two hours and
engineers could not reach the servers"
```

5. Componente de búsqueda semántica

El buscador (`src/semantic_search.py`) ordena los 60 documentos por similitud coseno con una consulta en lenguaje natural; con stemming y expansión basada en la ontología pone un documento relevante en primer lugar en las 8 consultas de prueba (MRR 1.00 contra 0.79 del TF-IDF simple).

Cómo funciona

1. **Recibe una consulta** en lenguaje natural, desde la línea de comandos, un modo interactivo o el conjunto `--demo` .
2. **Amplía la consulta con la ontología.** `expand_query()` busca las etiquetas alternativas SKOS definidas en `ontology/softtek_ontology.ttl` y agrega la etiqueta preferida del concepto y la de su concepto más amplio. Ejemplo: "went down" → + "outage incident"; "rules" → + "policy".
3. **Vectoriza** la consulta ampliada con el mismo vectorizador TF-IDF ajustado a los documentos, para que consulta y documentos compartan vocabulario.
4. **Calcula la similitud coseno** entre el vector de la consulta q y cada vector de documento d.
5. **Muestra los k documentos más relevantes** (5 por defecto) con su puntaje, id, título y categoría, además de los términos de la consulta que reconoció.

```
\cos(q, d) = \frac{q \cdot d}{\lVert q \rVert \, \lVert d \rVert}
```

Un puntaje de 1 significa la misma distribución de términos; 0 significa ningún término en común. La opción `--baseline` desactiva stemming y expansión, lo que permite la comparación antes/después de la sección 6.

Ejecución

```
python src/semantic_search.py "how do I deploy a service to kubernetes"
python src/semantic_search.py --demo
python src/semantic_search.py            # modo interactivo
```

Limitación. TF-IDF es léxico: solo encuentra palabras que aparecen en los documentos. La expansión con la ontología cubre los sinónimos registrados; los embeddings de oraciones (por ejemplo, sentence-transformers) son el siguiente paso natural para una búsqueda verdaderamente semántica. Los documentos de ejemplo están en inglés, por eso las consultas también.

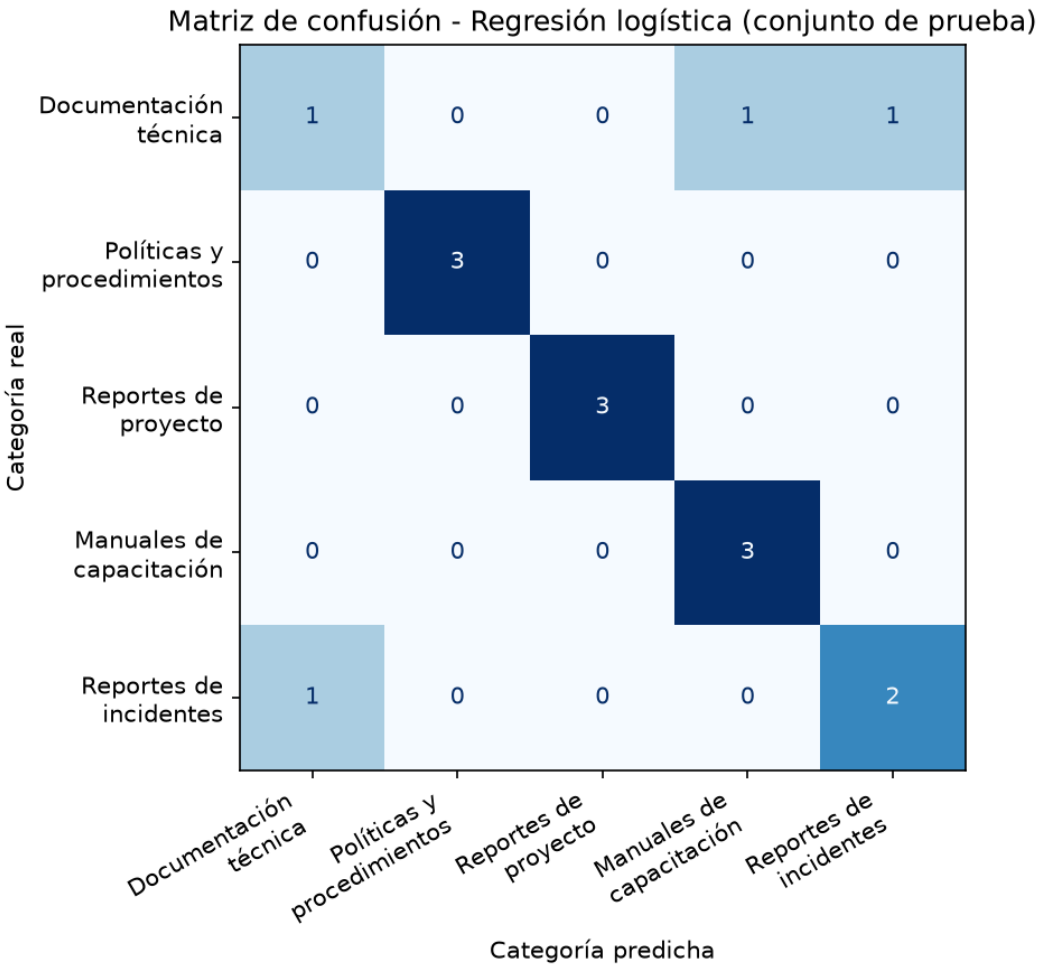
6. Evidencia del desarrollo del prototipo

Ambos componentes funcionan de principio a fin con la colección de 60 documentos: el clasificador etiqueta bien 5 de 6 documentos nuevos y el buscador devuelve primero el documento esperado en cada consulta de demostración. Las salidas de consola se copiaron tal cual de ejecuciones reales (guardadas en `outputs/`), por eso están en inglés.

6.1 Resultados de clasificación

Modelo	Exactitud en prueba (15 docs)	Exactitud en validación cruzada de 5 particiones (60 docs)
Naive Bayes multinomial	0.667	0.817 ± 0.082
Regresión logística (seleccionado)	0.800	0.900 ± 0.122

El conjunto de prueba tiene solo 3 documentos por clase, así que un error mueve la exactitud 6.7 puntos; la validación cruzada es la cifra más confiable.



Políticas, Reportes de proyecto y Manuales de capacitación salen perfectos en la prueba. Los errores ocurren entre Documentación técnica y Reportes de incidentes, que comparten vocabulario (API, service, database, deploy).

Raíces más discriminantes aprendidas por categoría

```
Incident Reports          incid, caus, hour, detect, outag, root
Policies & Procedures     polici, approv, procedur, employe, compani, busi
Project Reports           project, client, budget, team, sprint, percent
Technical Documentation   key, api, version, schema, use, store
Training Manuals          cours, train, practic, learn, develop, exercis
```

Clasificación de documentos nuevos sin etiqueta (`data/new_documents.csv`)

Id	Texto (inicio)	Predicción	Confianza	¿Correcto?
N01	The checkout service was down for 25 minutes after a certificate expired…	Reportes de incidentes	0.61	Sí
N02	Employees must lock their screen when leaving the desk…	Políticas y procedimientos	0.32	Sí
N03	The search service exposes a GET /v1/search endpoint…	Documentación técnica	0.49	Sí
N04	Module 3 of the course teaches how to write SQL queries…	Manuales de capacitación	0.69	Sí
N05	The project is two weeks behind schedule. The budget spent is 70 percent…	Reportes de proyecto	0.76	Sí
N06	New team members should complete the onboarding lessons on Docker and Kubernetes…	Reportes de proyecto	0.34	No (Manuales de capacitación)

N06 falla porque "team", "client project" y "members" son términos fuertes de Reportes de proyecto. Las predicciones con confianza baja (menos de 0.40 aprox.) podrían enviarse a un revisor humano.

6.2 Resultados de la búsqueda semántica

Ejecución de ejemplo (`python src/semantic_search.py --demo` , se muestran los 3 primeros):

```
Indexed 60 documents, vocabulary of 2180 TF-IDF terms.

Query: "how do I deploy a service to kubernetes"
  1. [0.322] TD03 - Kubernetes Deployment Guide (Technical Documentation)
  2. [0.155] TM03 - Introduction to Kubernetes Training (Training Manuals)
  3. [0.133] TD02 - Microservices Architecture Overview (Technical
```

```
Documentation)

Query: "what happened when the payment system went down"
Matched terms: incid, outag, payment, went
  1. [0.197] IR01 - Post-mortem: Payments API Outage (Incident Reports)
  2. [0.195] IR11 - Incident Report: Third-Party Email Service Outage
(Incident Reports)
  3. [0.168] IR04 - Incident Report: Data Center Network Failure (Incident
Reports)

Query: "rules for working from home"
  1. [0.426] PP02 - Remote Work Policy (Policies & Procedures)
  2. [0.092] PP01 - Information Security Policy (Policies & Procedures)
  3. [0.071] PR11 - Sprint 22 Retrospective - Payments Team (Project Reports)

Query: "course to learn security and avoid phishing"
  1. [0.217] TM12 - Information Security Awareness Training (Training
Manuals)
  2. [0.168] TM02 - Git Fundamentals Course (Training Manuals)
  3. [0.136] IR02 - Security Incident: Phishing Campaign (Incident Reports)

Query: "is the project over budget"
  1. [0.223] PR10 - Budget Variance Report - Mobile Banking (Project Reports)
  2. [0.177] PR05 - Cloud Migration Project Closure Report (Project Reports)
  3. [0.167] PR03 - Lessons Learned - Banking Portal Project (Project
Reports)
```

Antes y después de la mejora. Con TF-IDF simple (`--baseline`), la consulta "what happened when the payment system went down" solo reconoció *payment* y *went* y puso primero un post-mortem de caché; el reporte de la caída de la API de pagos ni siquiera apareció, porque el documento dice "Payments" y "Outage". El stemming (payments → payment) más el sinónimo de la ontología "went down" → "outage incident" lo subieron al lugar 1.

6.3 Capturas de pantalla del sistema en funcionamiento

Capturas reales de la Terminal de macOS ejecutando el prototipo (31 de octubre de 2026). Cada una muestra arriba el comando ejecutado. También están en el repositorio, en `docs/capturas/`.

Figura 1. Entrenamiento y evaluación del clasificador: Naive Bayes contra regresión logística.

Proyect SYADC — -zsh — 140×55

```
$ python src/classifier.py
Loaded 60 documents in 5 categories:
category
Technical Documentation    12
Policies & Procedures      12
Project Reports            12
Training Manuals           12
Incident Reports           12

Training documents: 45 | Test documents: 15

======================================================================
Naive Bayes: hold-out accuracy = 0.667 | 5-fold CV accuracy = 0.817 ± 0.082
======================================================================
                         precision    recall  f1-score   support

Technical Documentation       0.00      0.00      0.00         3
  Policies & Procedures       1.00      1.00      1.00         3
        Project Reports       0.75      1.00      0.86         3
       Training Manuals       0.75      1.00      0.86         3
       Incident Reports       0.33      0.33      0.33         3

               accuracy                           0.67        15
              macro avg       0.57      0.67      0.61        15
           weighted avg       0.57      0.67      0.61        15

Confusion matrix saved to outputs/confusion_matrix_naive_bayes.png

======================================================================
Logistic Regression: hold-out accuracy = 0.800 | 5-fold CV accuracy = 0.900 ± 0.122
======================================================================
                         precision    recall  f1-score   support

Technical Documentation       0.50      0.33      0.40         3
  Policies & Procedures       1.00      1.00      1.00         3
        Project Reports       1.00      1.00      1.00         3
       Training Manuals       0.75      1.00      0.86         3
       Incident Reports       0.67      0.67      0.67         3

               accuracy                           0.80        15
              macro avg       0.78      0.80      0.78        15
           weighted avg       0.78      0.80      0.78        15

Confusion matrix saved to outputs/confusion_matrix_logistic_regression.png

Most discriminative terms per category:
  Incident Reports          incid, caus, hour, detect, outag, root
```

Figura 2. Clasificación de los 6 documentos nuevos y de un texto libre con `--text`.

Proyect SYADC — -zsh — 140×55

```
$ python src/classifier.py   (final model on new, unlabelled documents)

Classifying new, unlabelled documents (data/new_documents.csv):
  [N01] The checkout service was down for 25 minutes after a certificate expir...
        -> Incident Reports (confidence 0.61)
  [N02] Employees must lock their screen when leaving the desk and must not sh...
        -> Policies & Procedures (confidence 0.32)
  [N03] The search service exposes a GET /v1/search endpoint that accepts a qu...
        -> Technical Documentation (confidence 0.49)
  [N04] Module 3 of the course teaches how to write SQL queries. Complete the ...
        -> Training Manuals (confidence 0.69)
  [N05] The project is two weeks behind schedule. The budget spent is 70 perce...
        -> Project Reports (confidence 0.76)
  [N06] New team members should complete the onboarding lessons on Docker and ...
        -> Project Reports (confidence 0.34)

Results saved to outputs/new_documents_classified.csv

$ python src/classifier.py --text "The VPN was unavailable for two hours and engineers could not reach the servers"
Text: The VPN was unavailable for two hours and engineers could not reach the servers
-> Incident Reports (confidence 0.39)
```

Figura 3. Búsqueda semántica con las 5 consultas de demostración.

Proyect SYADC — -zsh — 140×55

```
$ python src/semantic_search.py --demo -k 3
Indexed 60 documents, vocabulary of 2180 TF-IDF terms.

Query: "how do I deploy a service to kubernetes"
Matched terms: deploy, deploy servic, kubernet, servic, servic kubernet
  1. [0.322] TD03 — Kubernetes Deployment Guide (Technical Documentation)
  2. [0.155] TM03 — Introduction to Kubernetes Training (Training Manuals)
  3. [0.133] TD02 — Microservices Architecture Overview (Technical Documentation)

Query: "what happened when the payment system went down"
Matched terms: incid, outag, payment, went
  1. [0.197] IR01 — Post-mortem: Payments API Outage (Incident Reports)
  2. [0.195] IR11 — Incident Report: Third-Party Email Service Outage (Incident Reports)
  3. [0.168] IR04 — Incident Report: Data Center Network Failure (Incident Reports)

Query: "rules for working from home"
Matched terms: home, polici, remot, remot work, rule, work, work polici
  1. [0.426] PP02 — Remote Work Policy (Policies & Procedures)
  2. [0.092] PP01 — Information Security Policy (Policies & Procedures)
  3. [0.071] PR11 — Sprint 22 Retrospective — Payments Team (Project Reports)

Query: "course to learn security and avoid phishing"
Matched terms: avoid, cours, cours learn, learn, phish, secur, train
  1. [0.217] TM12 — Information Security Awareness Training (Training Manuals)
  2. [0.168] TM02 — Git Fundamentals Course (Training Manuals)
  3. [0.136] IR02 — Security Incident: Phishing Campaign (Incident Reports)

Query: "is the project over budget"
Matched terms: budget, project
  1. [0.223] PR10 — Budget Variance Report — Mobile Banking (Project Reports)
  2. [0.177] PR05 — Cloud Migration Project Closure Report (Project Reports)
  3. [0.167] PR03 — Lessons Learned — Banking Portal Project (Project Reports)
```

Figura 4. La misma consulta con TF-IDF simple y con stemming + expansión con la ontología.

Proyect SYADC — -zsh — 140×55

```
$ python src/semantic_search.py --baseline -k 3 "what happened when the payment system went down"   (plain TF-IDF)
Indexed 60 documents, vocabulary of 2369 TF-IDF terms.

Query: "what happened when the payment system went down"
Matched terms: payment, went
  1. [0.103] IR10 — Post-mortem: Cache Stampede on Product Catalog (Incident Reports)
  2. [0.098] PR01 — Retail Platform Project Status Report — March (Project Reports)

$ python src/semantic_search.py -k 3 "what happened when the payment system went down"   (stemming + ontology expansion)
Indexed 60 documents, vocabulary of 2180 TF-IDF terms.

Query: "what happened when the payment system went down"
Matched terms: incid, outag, payment, went
  1. [0.197] IR01 — Post-mortem: Payments API Outage (Incident Reports)
  2. [0.195] IR11 — Incident Report: Third-Party Email Service Outage (Incident Reports)
  3. [0.168] IR04 — Incident Report: Data Center Network Failure (Incident Reports)
```

Figura 5. Evaluación de la búsqueda (P@3, MRR) y consultas SPARQL a la ontología.

Proyect SYADC — -zsh — 140×55

```
$ python exercises/ex5_retrieval_evaluation.py
Query                                              RR base   RR new
how do I deploy a service to kubernetes               1.00     1.00
what happened when the payment system went down       0.00     1.00
rules for working from home                           1.00     1.00
course to learn security and avoid phishing           0.33     1.00
is the project over budget                            1.00     1.00
login tokens and identity provider                    1.00     1.00
the website broke after a release                     1.00     1.00
who approves changes to production                    1.00     1.00

Baseline TF-IDF                          P@3 = 0.375   MRR = 0.792
TF-IDF + stemming + ontology expansion   P@3 = 0.417   MRR = 1.000

$ python exercises/ex6_ontology_queries.py
Ontology loaded: 372 triples

Q1. Which document types exist?
    Incident Reports
    Policies & Procedures
    Project Reports
    Technical Documentation
    Training Manuals

Q2. Which documents relate to the Payments API (directly or through an incident)?
    IR01 | Post-mortem: Payments API Outage | documents an incident on it
    TD01 | Payments API Reference v3 | describes the system

Q3. Which documents must be updated because of lessons learned from incidents?
    Payments API outage (47 min) | Automate TLS certificate renewal | Logging and Monitoring Setup
    Inventory DB degradation | Add performance test after migrations | Database Schema for Inventory Service
    Inventory DB degradation | Add performance test after migrations | Change Management Procedure

Q4. What should a new SRE read to operate Kubernetes? (skill -> training -> referenced docs)
    Introduction to Kubernetes Training | Kubernetes Deployment Guide

Q5. How many documents does each department own?
    Cloud Operations | 6
    Engineering | 4
    People & Learning | 4
    Project Management Office | 2
    Compliance | 1
```

7. Ejercicios prácticos (Temas 1–6)

Seis ejercicios en `exercises/` construyeron las piezas que reutiliza el prototipo; cada uno se ejecuta por separado y su salida se guarda en `outputs/ex*.txt`.

#	Ejercicio	Área del tema	Resultado clave
1	Preprocesamiento de texto: tokens, stop words, stemming de Porter	Representación de texto	Vocabulario de 986 palabras → 704 raíces (29% menor)
2	TF-IDF calculado a mano con 3 documentos de juguete	Modelo de espacio vectorial	Los valores manuales coinciden exactamente con scikit-learn (`np.allclose` = True)
3	Similitud coseno entre documentos	Medidas de similitud	La similitud dentro de una categoría es 2–5× mayor que entre categorías
4	Pruebas de clasificación: 4 representaciones × 3 modelos	Clasificación automática	Mejor exactitud en validación cruzada: 0.90 (TF-IDF + regresión logística / SVM lineal)
5	Evaluación de recuperación con juicios de relevancia	Recuperación de información	MRR 0.79 → 1.00; P@3 0.375 → 0.417
6	Consultas SPARQL sobre la ontología	Representación conceptual	5 preguntas de competencia respondidas con 372 tripletas

Ejercicio 2 — TF-IDF a mano

```
term        df     idf   tf-idf in doc1 / doc2 / doc3
outage       2   1.288   0.322 / 0.000 / 0.373
security     2   1.288   0.000 / 0.402 / 0.373
server       1   1.693   0.847 / 0.000 / 0.000
restart      1   1.693   0.423 / 0.000 / 0.000
Manual result equals scikit-learn: True
```

Los términos que aparecen en dos documentos (outage, security) tienen un idf menor que los que aparecen en uno; "server" aparece dos veces solo en doc1, por eso obtiene el peso más alto.

Ejercicio 3 — Documentos más similares

```
0.416  TD10 Caching Strategy for Product Catalog  <->  IR10 Post-mortem:
Cache Stampede on Product Catalog
0.416  TD12 Message Queue Integration Guide       <->  IR07 Post-mortem:
Message Queue Backlog
0.317  PP04 Data Privacy Policy                   <->  IR09 Incident:
Accidental Data Exposure
0.307  TD03 Kubernetes Deployment Guide           <->  TM03 Introduction to
Kubernetes Training
```

Los vínculos más fuertes cruzan tipos de documento alrededor del mismo sistema, que es justo la relación documento–concepto modelada en la ontología (un reporte de incidente referencia la documentación técnica del sistema afectado). También explica las confusiones entre Documentación técnica y Reportes de incidentes de la sección 6.

Ejercicio 4 — Pruebas de clasificación (exactitud en validación cruzada de 5 particiones)

Representación	Naive Bayes	Regresión logística	SVM lineal
Bolsa de palabras (conteos)	0.850	0.850	0.883
TF-IDF unigramas	0.817	0.883	0.900
TF-IDF + stemming	0.850	0.883	0.883
TF-IDF + stemming + bigramas	0.817	0.900	0.867

Las diferencias están dentro de una desviación estándar (±0.10 aprox.) con 60 documentos, así que el prototipo conserva la regresión logística, simple e interpretable, con stemming y bigramas.

Ejercicio 5 — Evaluación de la búsqueda (8 consultas evaluadas)

Configuración	P@3	MRR
TF-IDF base	0.375	0.792
TF-IDF + stemming + expansión con ontología	0.417	1.000

Mejoraron dos consultas: "payment system went down" (rango recíproco 0 → 1) y "course to learn security and avoid phishing" (0.33 → 1).

Ejercicio 6 — Consultas a la ontología (SPARQL)

```
Q2. Which documents relate to the Payments API?
    IR01 | Post-mortem: Payments API Outage | documents an incident on it
    TD01 | Payments API Reference v3        | describes the system
Q3. Which documents must be updated because of lessons learned?
    Payments API outage      | Automate TLS certificate renewal      |
Logging and Monitoring Setup
    Inventory DB degradation | Add performance test after migrations |
Database Schema for Inventory Service
    Inventory DB degradation | Add performance test after migrations | Change
Management Procedure
Q4. What should a new SRE read to operate Kubernetes?
    Introduction to Kubernetes Training | Kubernetes Deployment Guide
```

Q3 muestra en el grafo la etapa de actualización del ciclo de vida: los incidentes producen lecciones aprendidas que apuntan a los documentos que hay que revisar.

8. Conclusiones y siguientes pasos

El prototipo muestra que las técnicas básicas ya cubren el núcleo del KMS: TF-IDF + regresión logística clasifica los tipos de documento con 0.90 de exactitud, y la búsqueda TF-IDF mejora mucho cuando la ontología le aporta sinónimos.

Siguientes pasos:

- Ampliar la colección a más de 60 documentos y volver a evaluar; los puntajes actuales varían ±0.10 entre particiones.
- Enviar a un revisor humano las predicciones con confianza menor a 0.40 y usar sus correcciones como nuevos datos de entrenamiento.

- Agregar embeddings de palabras u oraciones para buscar por significado, con TF-IDF como respaldo.
- Poblar la ontología en automático: ligar cada documento nuevo con los sistemas y conceptos que menciona.

Archivos del proyecto

Todo el código, los documentos de prueba y las capturas están en https://github.com/regina120506/Avance_SYTAC.

Ruta	Contenido
`data/documents.csv`	60 documentos de ejemplo etiquetados
`data/new_documents.csv`	6 documentos sin etiqueta para la demostración de clasificación
`src/classifier.py`	Prototipo de clasificación (sección 4)
`src/semantic_search.py`	Componente de búsqueda semántica (sección 5)
`src/kms_utils.py`	Carga, preprocesamiento y expansión de consultas con la ontología
`ontology/softtek_ontology.ttl`	Ontología en OWL/SKOS Turtle (sección 3)
`exercises/ex1…ex6_*.py`	Ejercicios prácticos (sección 7)
`outputs/`	Registros de ejecución, matrices de confusión, mapa de calor, modelo guardado

Instalación: `python -m venv .venv && .venv/bin/pip install -r requirements.txt`.

Fuentes

- Softtek Fast Facts (2024) — fecha de fundación, empleados, oficinas, centros de entrega, servicios
- Softtek — Wikipedia — historia y fundador